MANCHESTER 1824
The University of Manchester

AERO60492 – Autonomous Mobile Robots

Lecture 10 – Feedback Control

pawel.ladosz@manchester.ac.uk

Pre-coursework 3 announcements

- Coursework will have you design, tune and test (both in simulation and in experiment) controller for UAV position
- The coursework has two components simulation and the lab
- Groups and coursework sheet will be released Monday next week
- First session will be Monday next week, I will introduce coursework then
- Please make sure you do not miss it

Feedback Discussion

- **This module is too easy** – Last year AMR had a high failure rate in exam (probably the highest of all robotics courses). A lot of marks was lost due to basic mathematic errors. Perhaps course is a little bit unbalanced, with more difficult content in second half of semester.
- **Make sure lectures are uploaded before the delivery** – that was clerical error on our part, shouldn't happen again.
- Keep the feedback coming, thanks! (especially about activities!

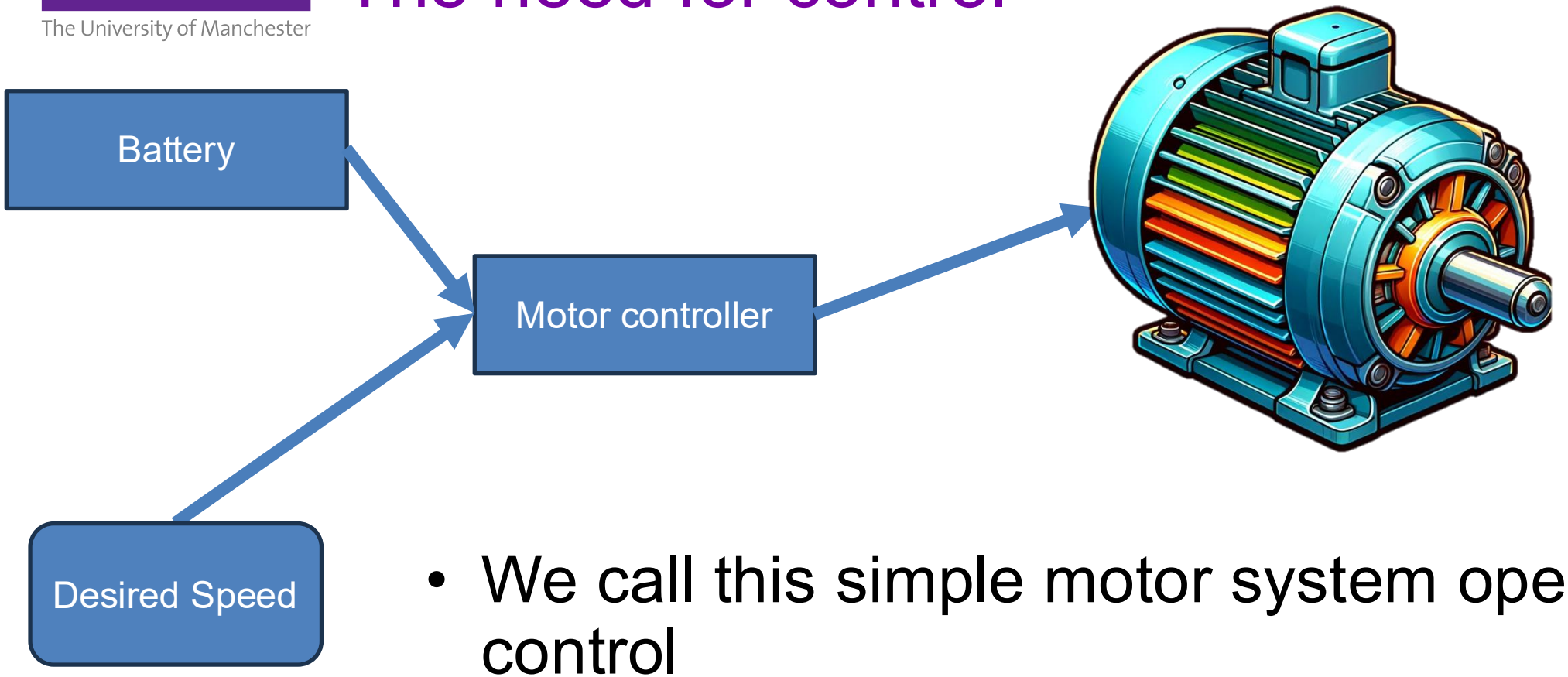
Today's session

- 1.5 hours of lecture
- 0.5 hours of activity

Intended learning outcomes

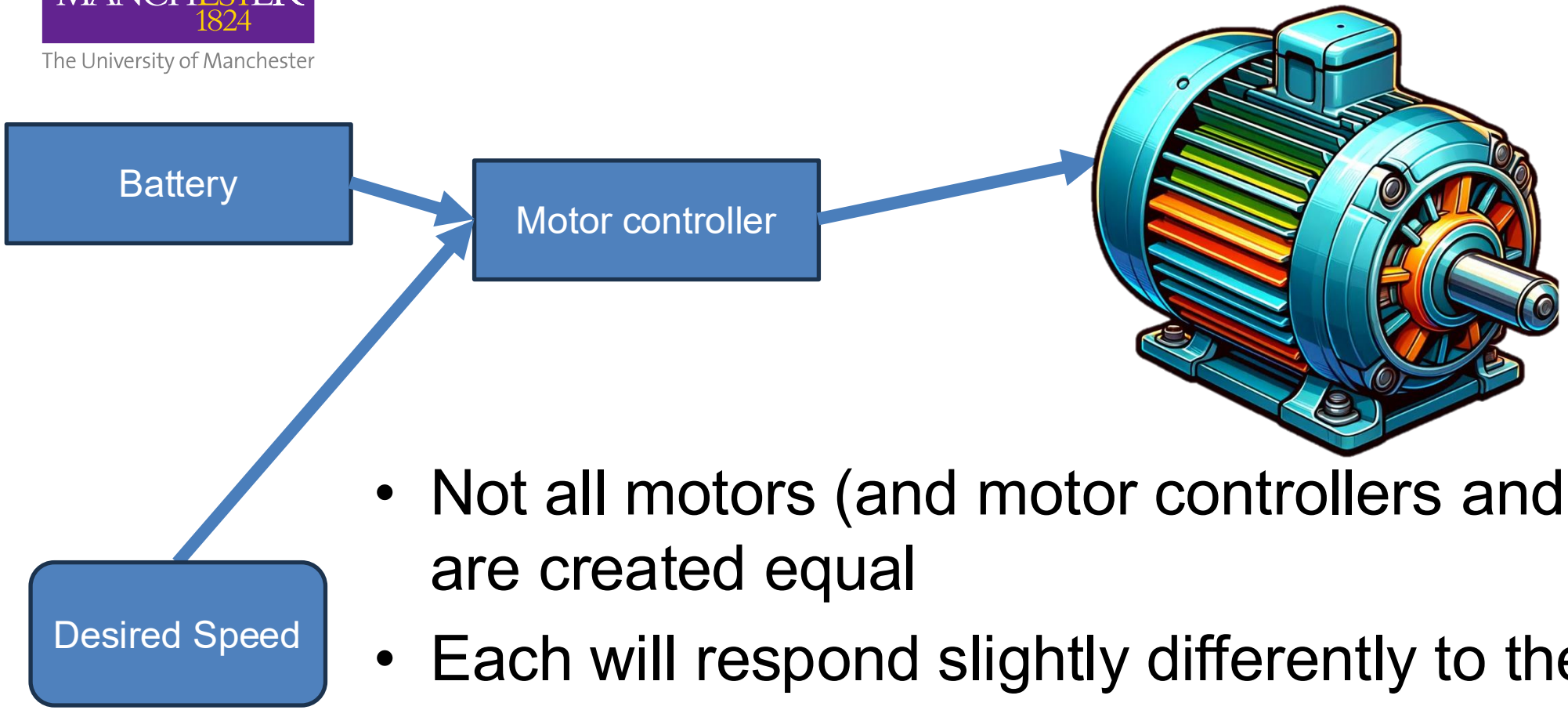
- Understanding the need for controllers
- Understand and use PID controllers
- Use tuning methods to correctly tune the PID controller
- Appreciate the existence and the need for more advanced methods

The need for control



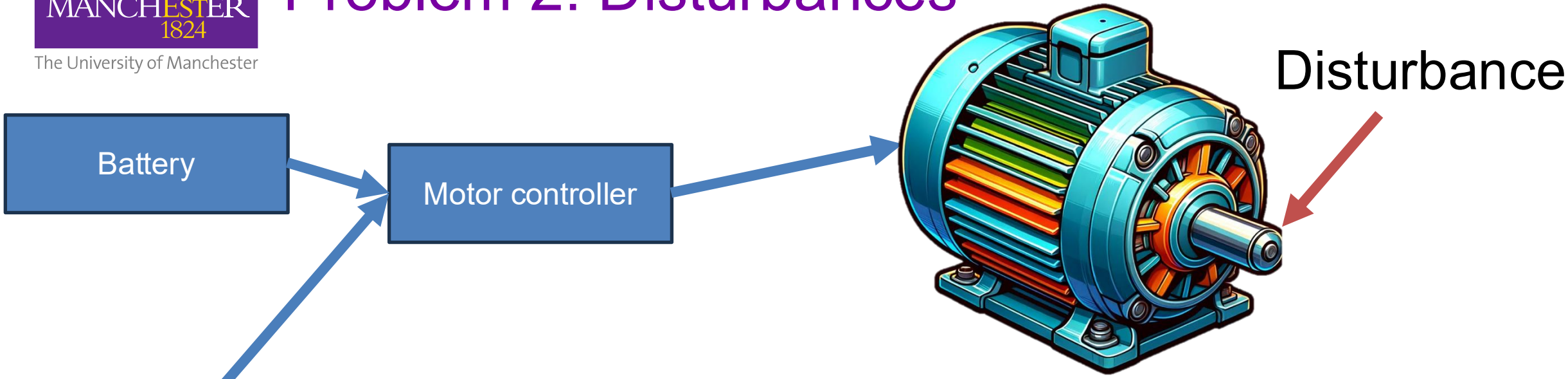
- We call this simple motor system open-loop control
- Issue: The speed you desire vs the speed you actually get

Problem 1: uncertain system characteristics



- Not all motors (and motor controllers and battery) are created equal
- Each will respond slightly differently to the same voltage input
- We could adjust the signal strength level for each individual system – time consuming and difficult

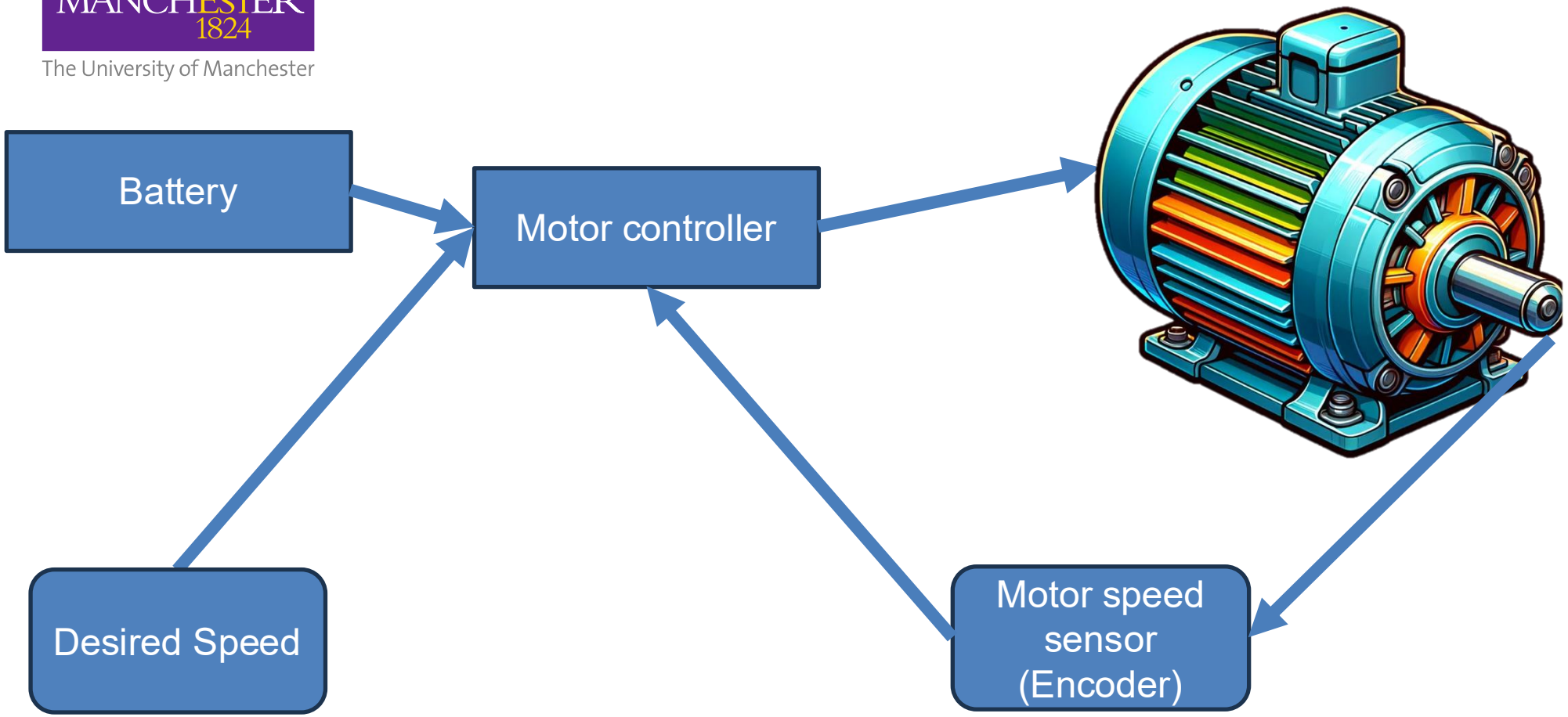
Problem 2: Disturbances



Desired Speed

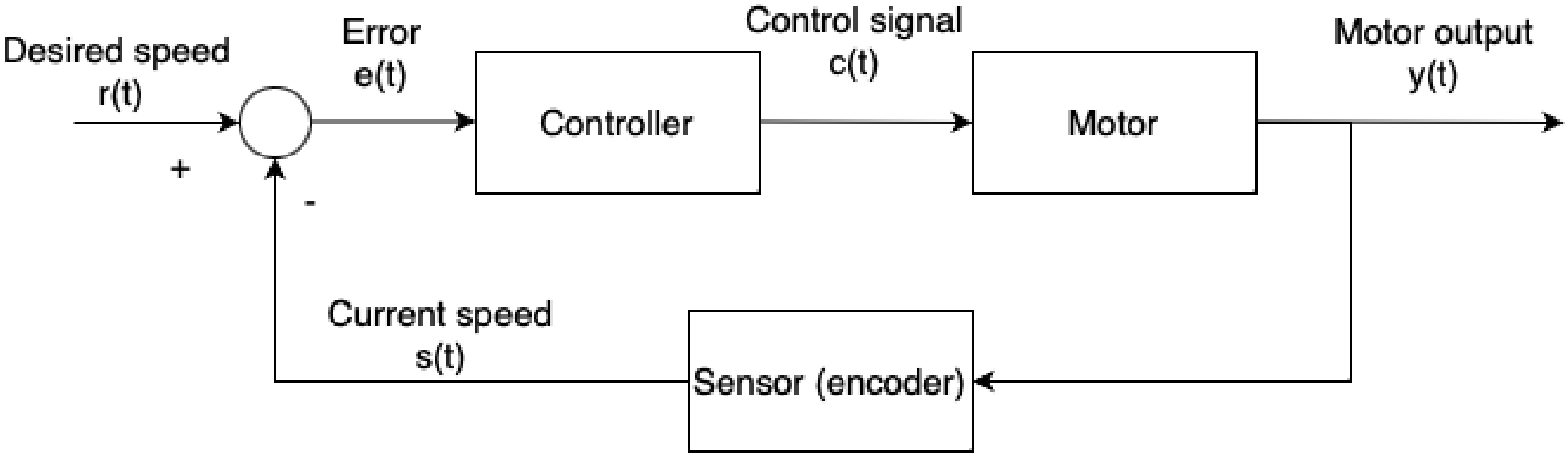
- Imagine somebody is holding the output of your motor
- Your signal will result in a different speed
- Thus the need for feedback controller arises

Feedback controller introduction

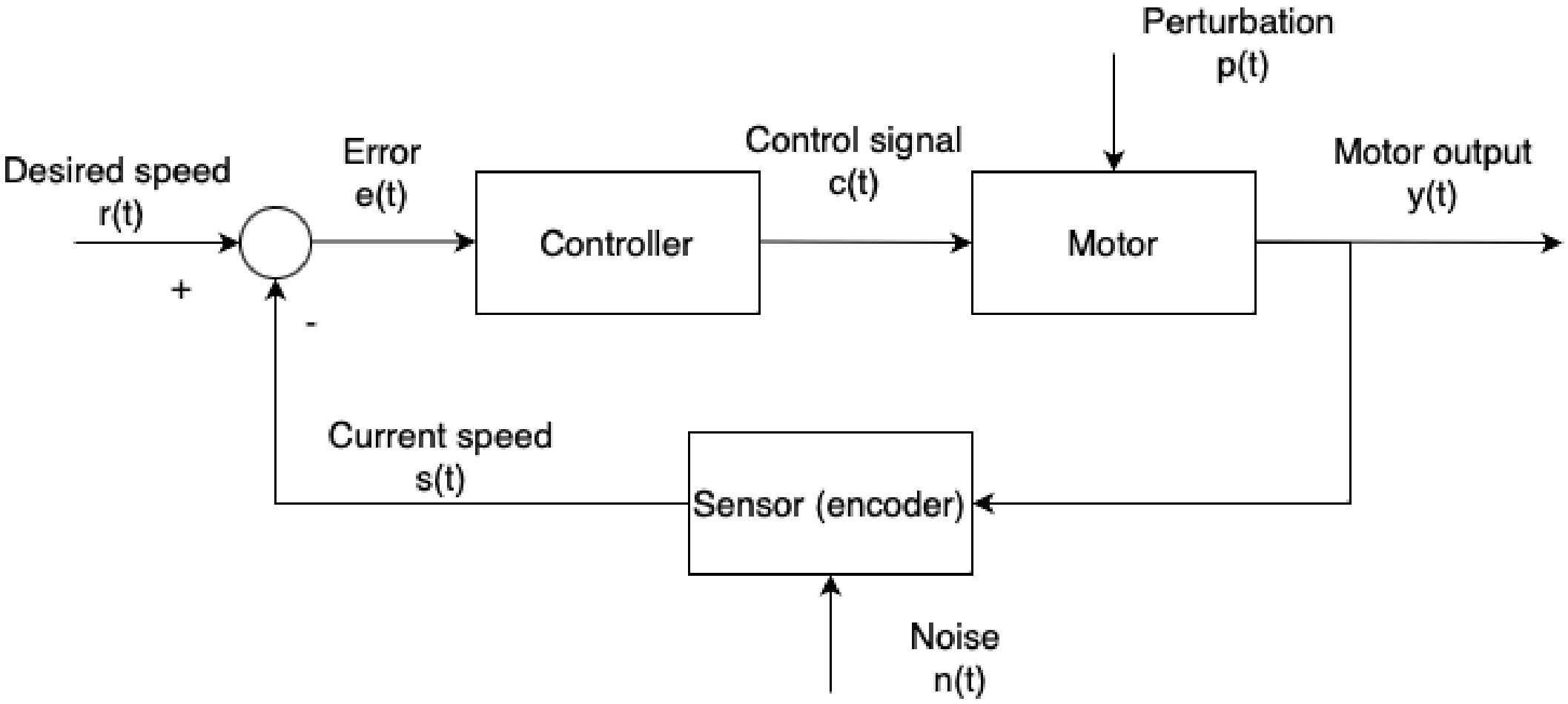


- Use sensor to tell us what the motor's speed is and correct it based on desired speed! (feedback control or closed-loop control)

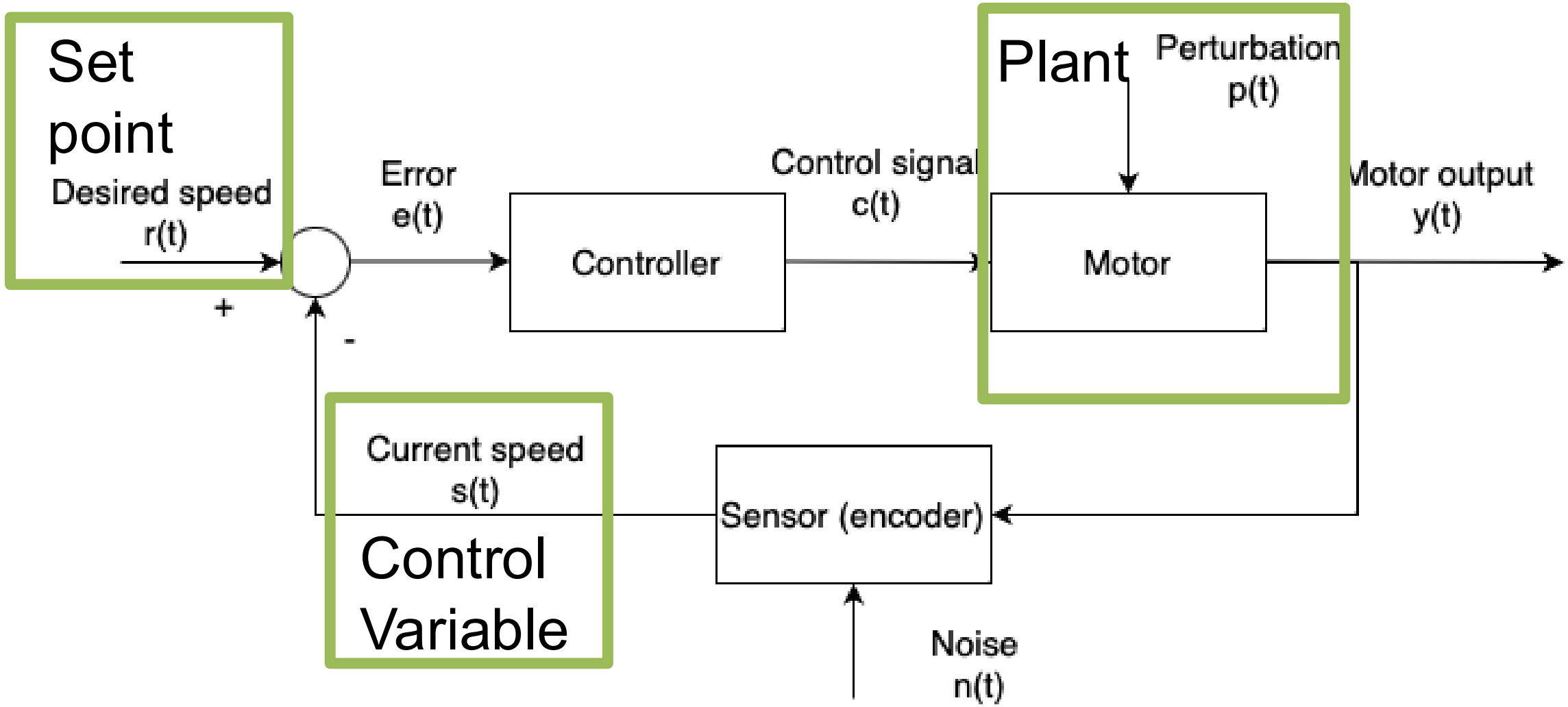
Convert the system to more commonly used block diagram



Add noise

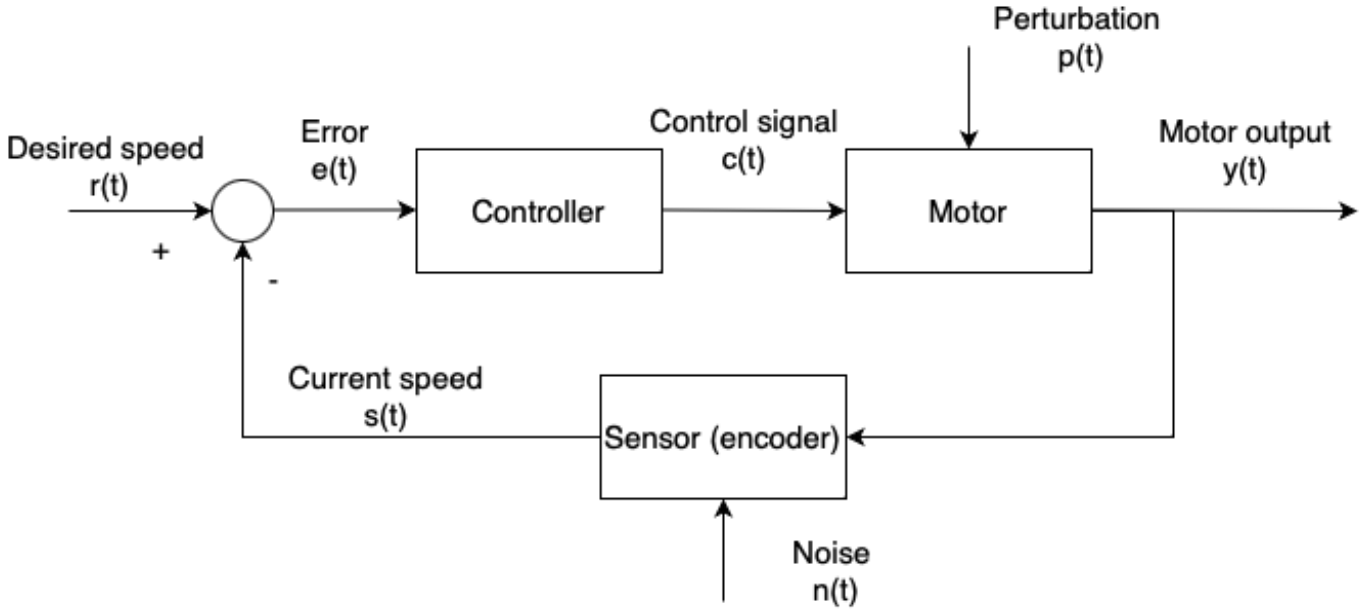

Perturbation
p(t)
Desired speed
r(t)
Error
e(t)
Control signal
c(t)
Motor output
y(t)
Controller
Motor
+
-
Current speed
s(t)
Sensor (encoder)
Noise
n(t)

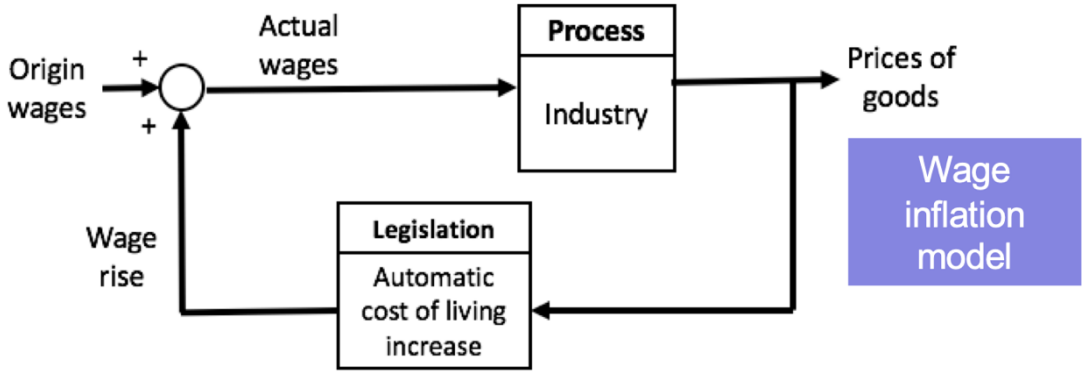
Some more terminology



Negative vs positive feedback

- The system presented so far is negative feedback i.e. control variable is subtracted from set point
- they are said to be **stable** as they converge towards a solution (0 error)

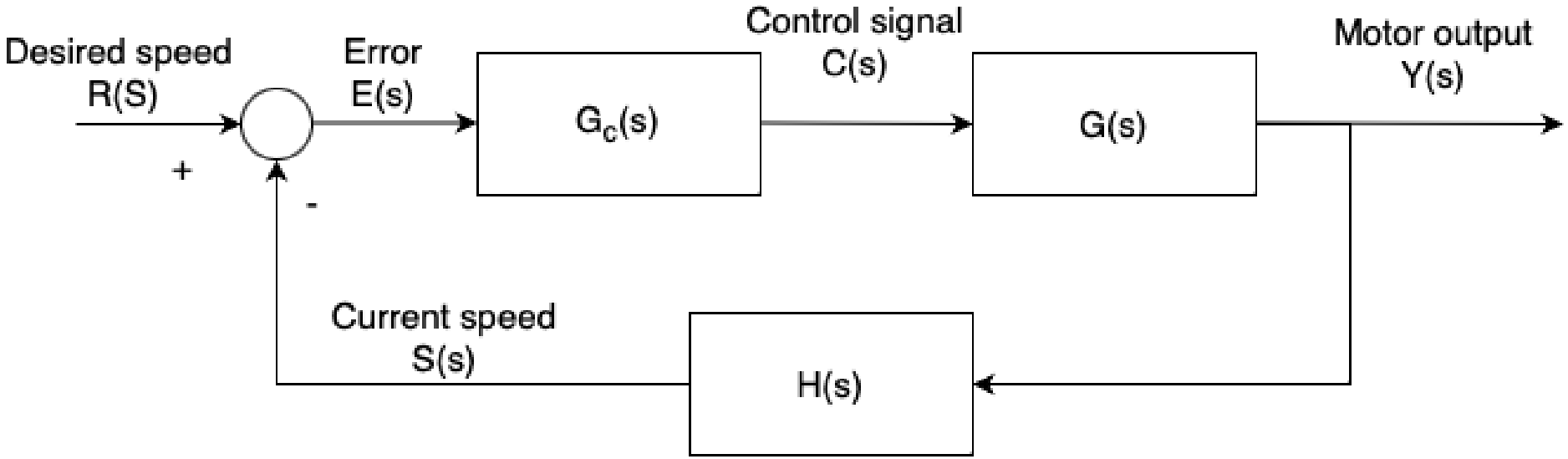




- Positive feedback system is when control variable is added to setpoint
- they are said to be **unstable** as they grow infinitely

Laplace transformation

- Systems are often represented in Laplace transformed way
  - This allows easier manipulation and solution of block diagram
- Thus our simple system in Laplace domain becomes:



Linear vs non-linear systems

- Linear systems:
  - If I increase control signal by x, the output increase y is proportional (linearly) to x
  - Linear systems satisfy superposition principle

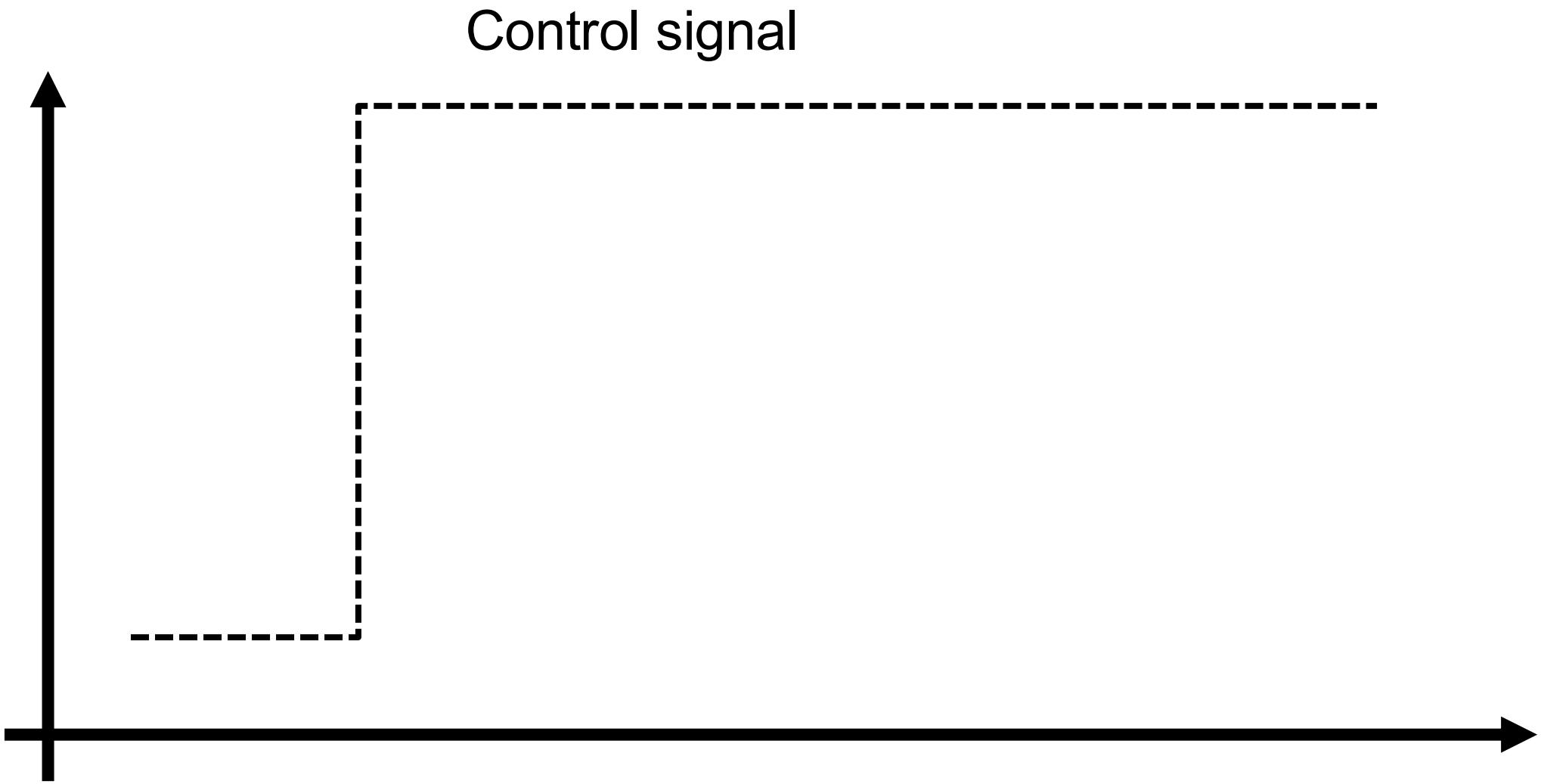
$$F(x_1 + x_2) = F(x_1) + F(x_2),\ F(ax_1) = aF(x_1)$$

- Non-linear systems
  - If I increase control signal by x the increase in output is difficult to predict – controllers are difficult!
- Often we approximate non-linear systems by linear systems to allow controller to work

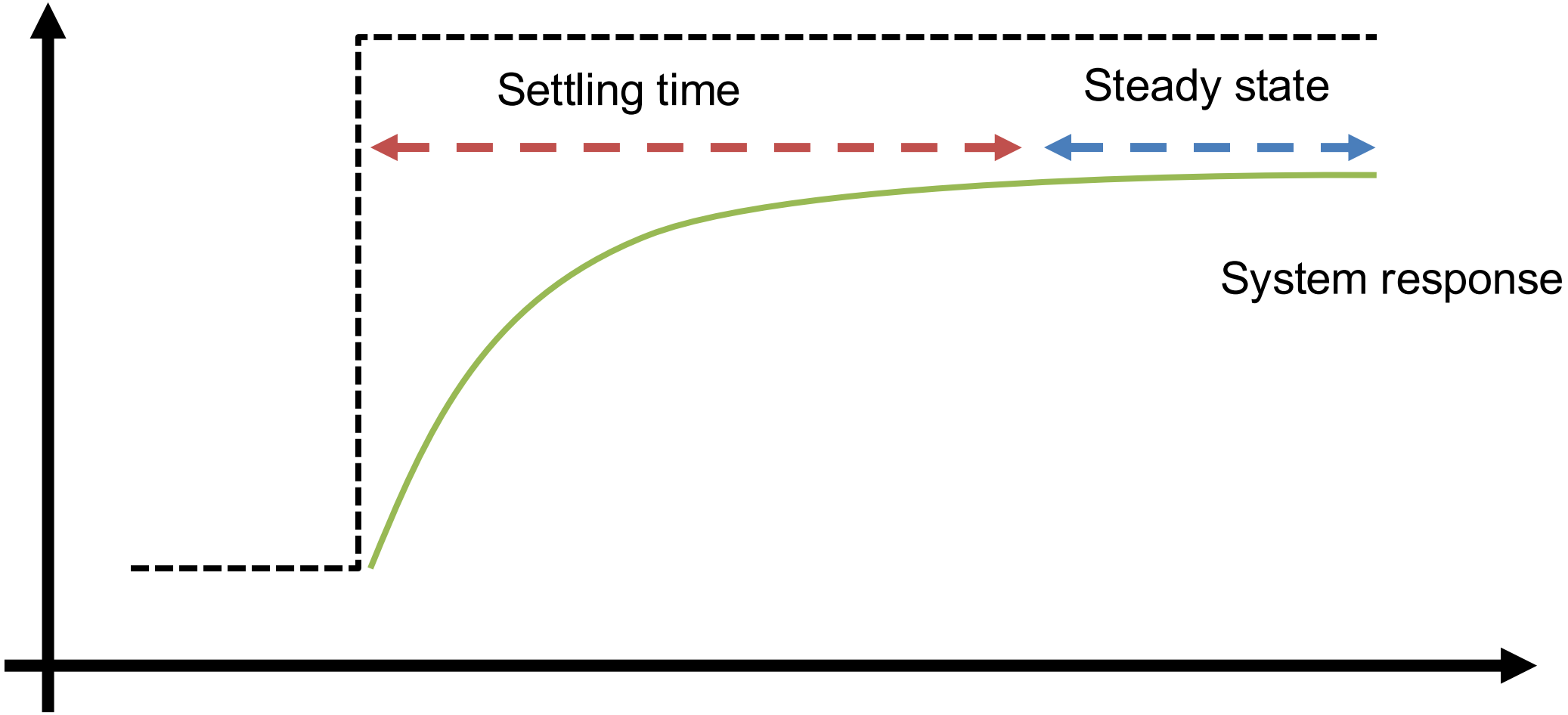
MANCHESTER 1824
The University of Manchester

Control signal and responses

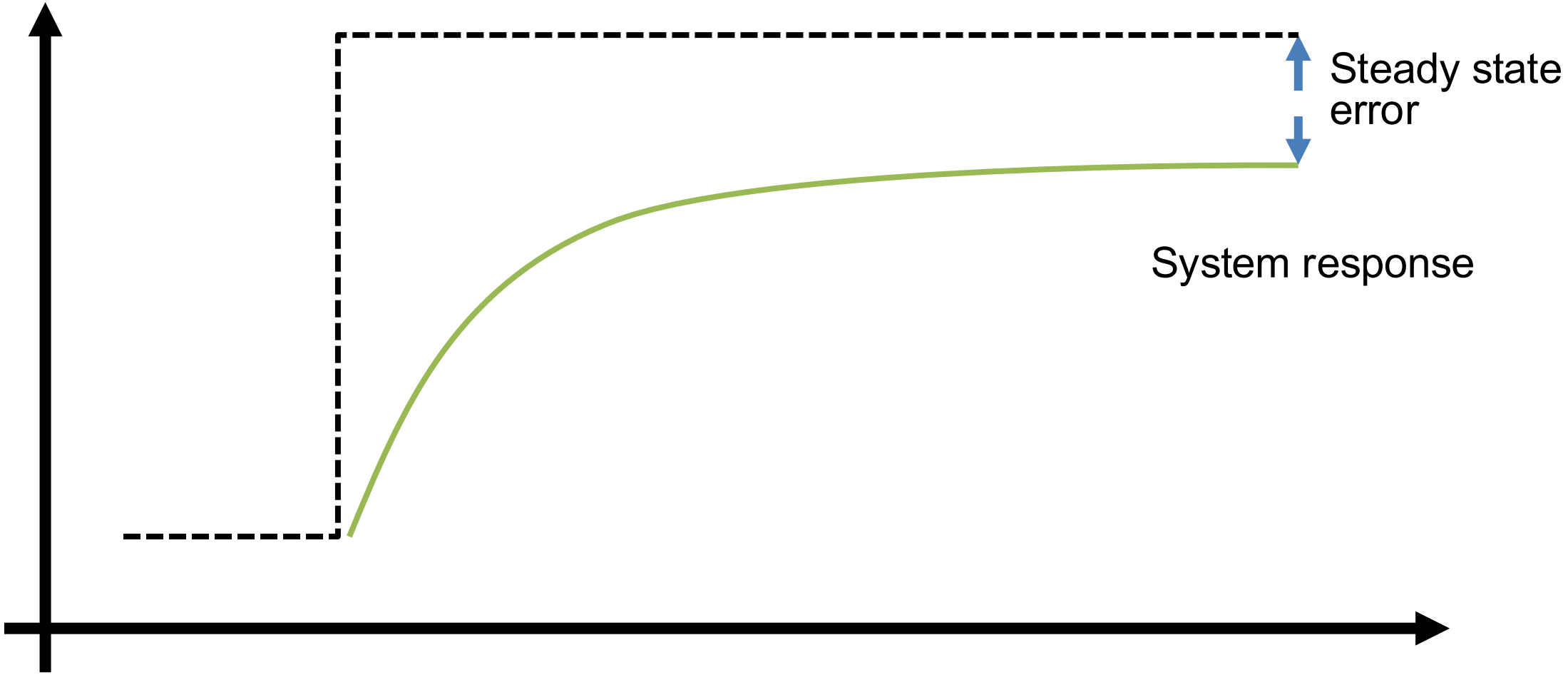
Control signal inputs



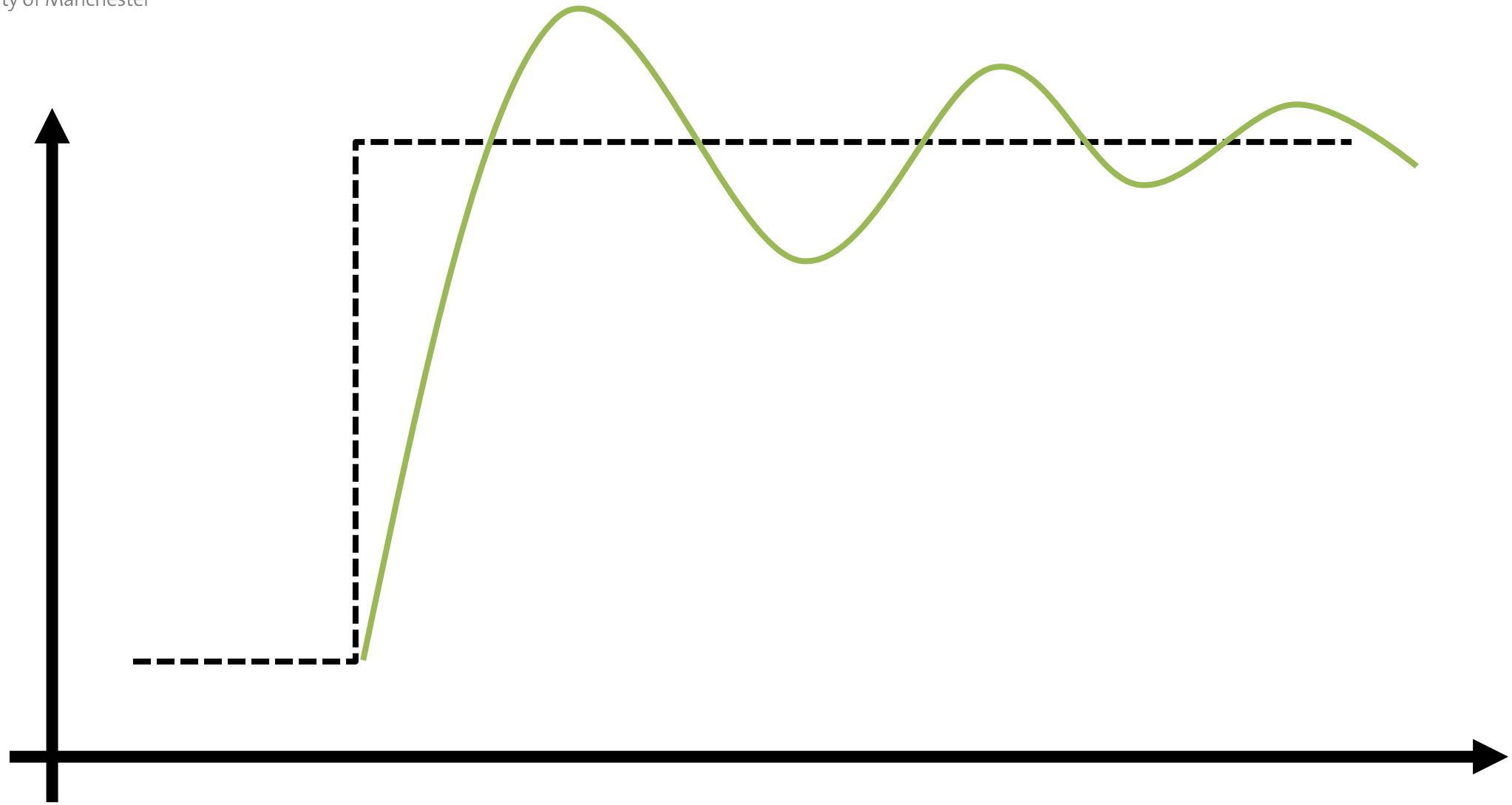
General signal properties



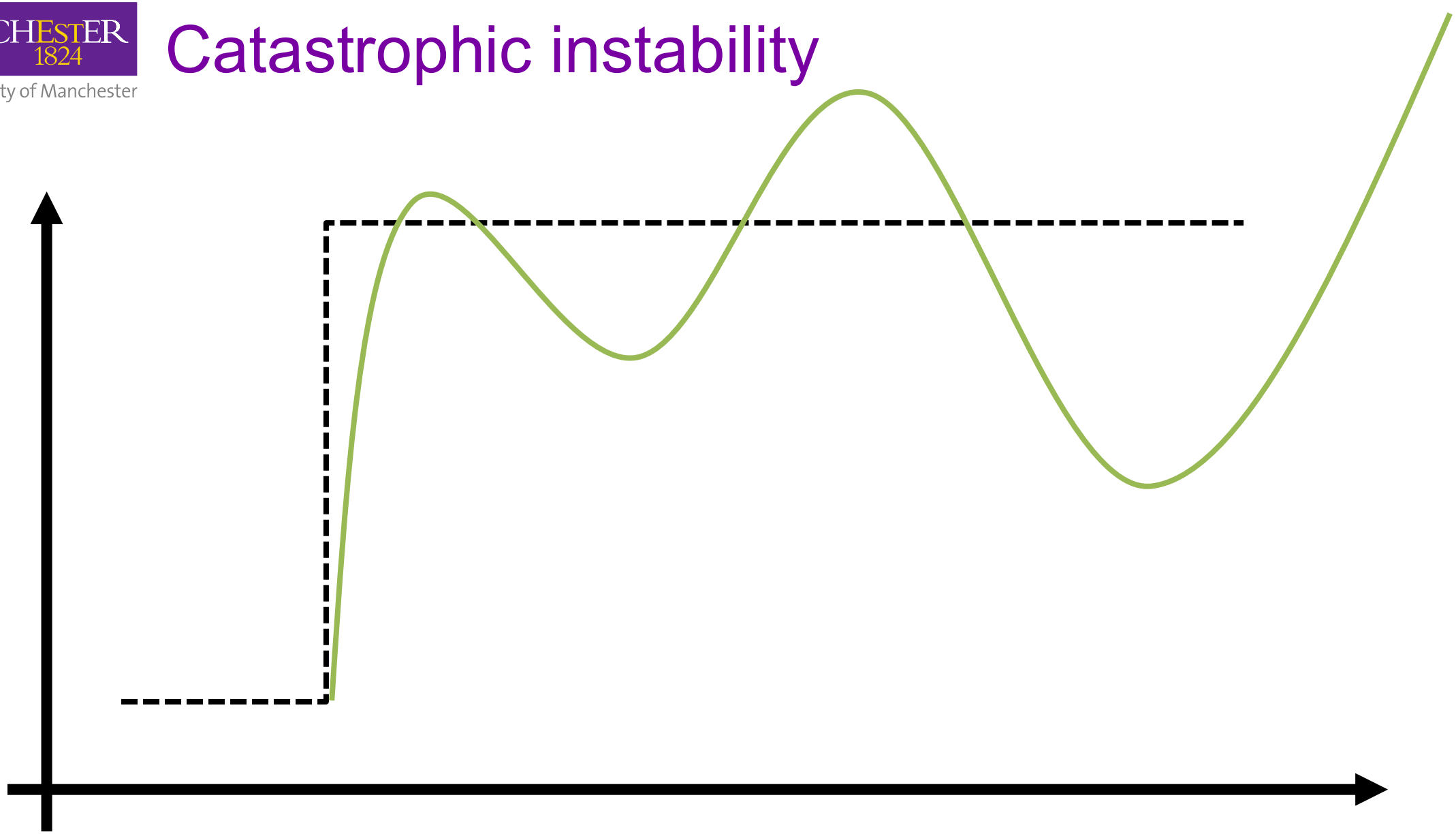
Steady state error



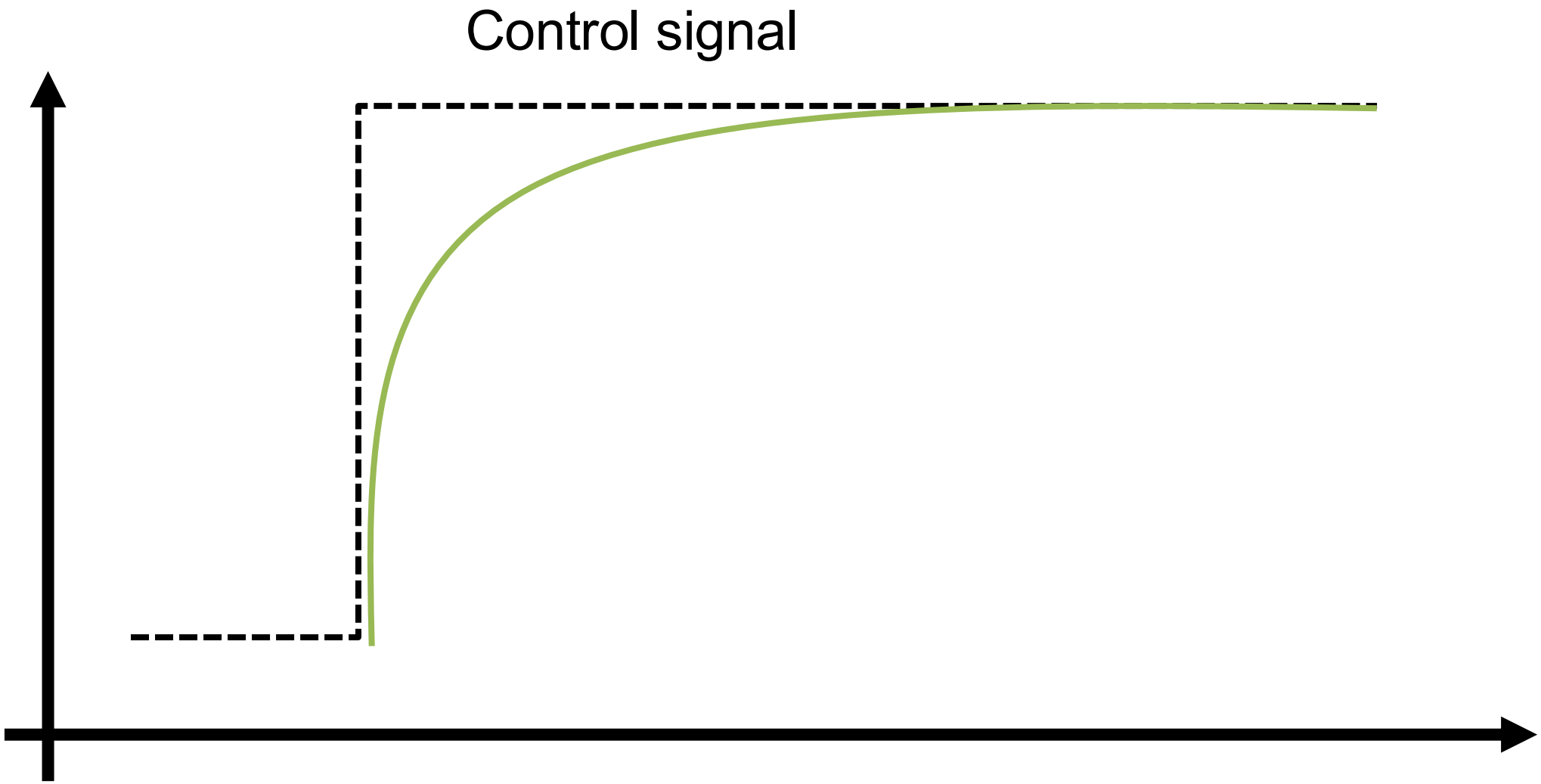
Oscillation (underdamped)



Catastrophic instability



Desired system (critically damped)

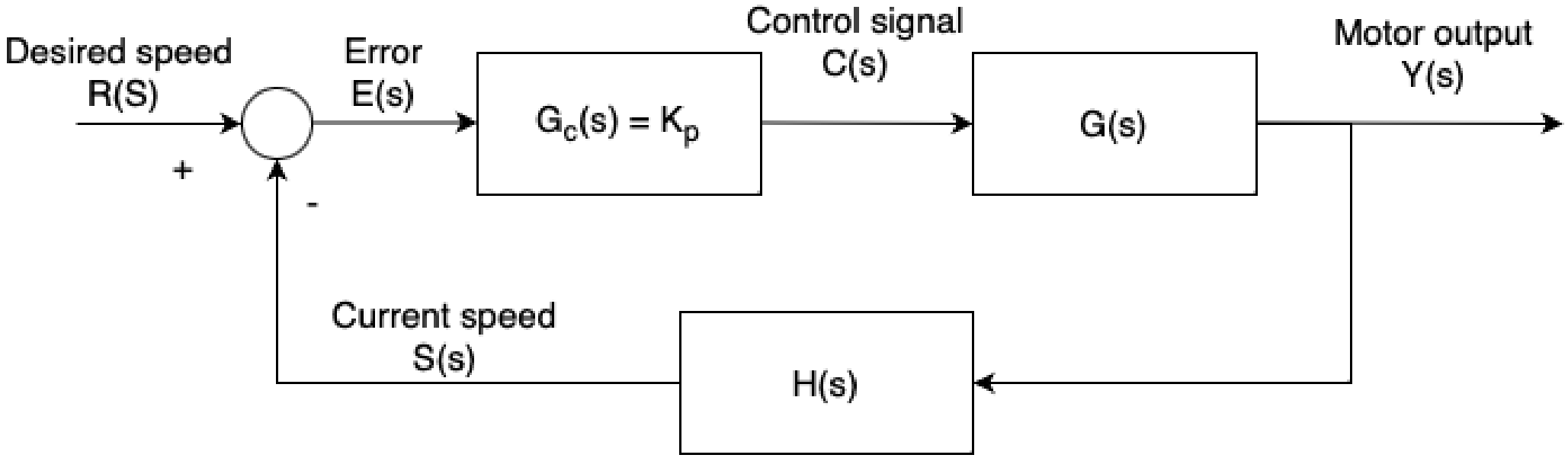


MANCHESTER 1824
The University of Manchester

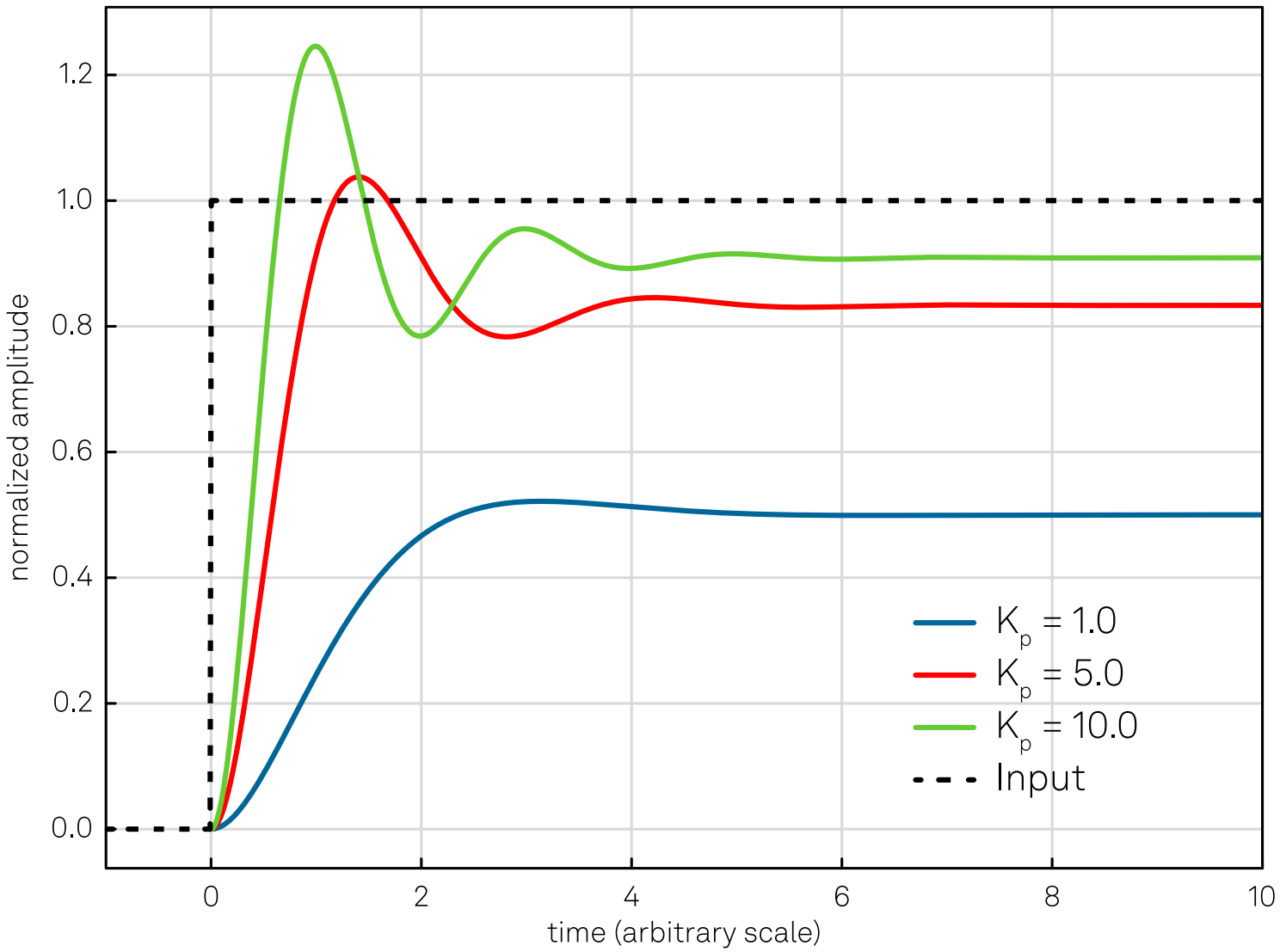
PID controller

P-controller

- The simplest type of controller
- Simply multiply the error by a constant:

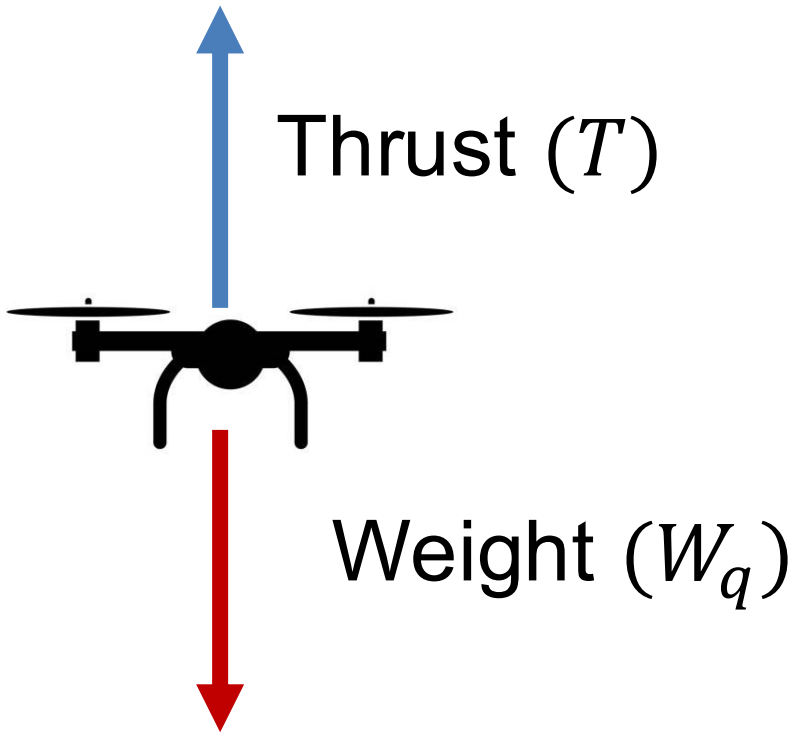


Issues with P-only controller



Why steady state error with P-controller?

- Consider simple system where we control thrust of motors



- The task is to balance Quadrotor at a certain height *h*

Why steady state error with P-controller?

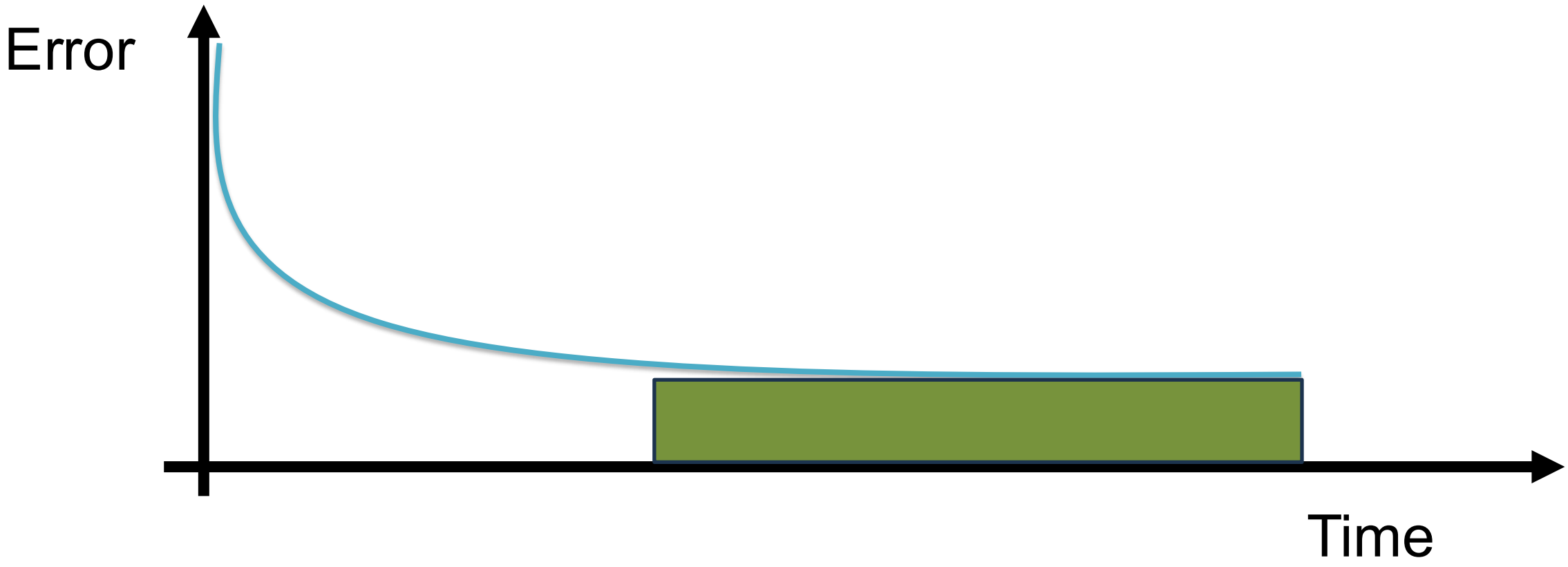
- Suppose that $K_p = 10, W_q = 10\ N, \mathrm{h} = 10\ \mathrm{m}$ and initial height is $1\ m$
- Generally, we can control thrust based on height as: $T_d = (h_d - h_c) * K_p$
- Note for quadcopter to increase height $W_q < \mathrm{T}$
- Demanded Trust then becomes:
  - $T_d = (10 - 1) * 10 = 90\ N$
  - $T_d = (10 - 5) * 10 = 50\ N$
  - $T_d = (10 - 9) * 10 = 10\ N = W_q$! – unable to fly any higher, reached steady state with 1 meter error

What happens if we increase P-gain?

- We could increase P-gain to alleviate this problem
- This leads to oscillation instability due to system dynamics!
  - Quadcopter is unable to stop in a single spot, thus it will overshoot the set point, introducing oscillation and in extreme cases catastrophic oscillation

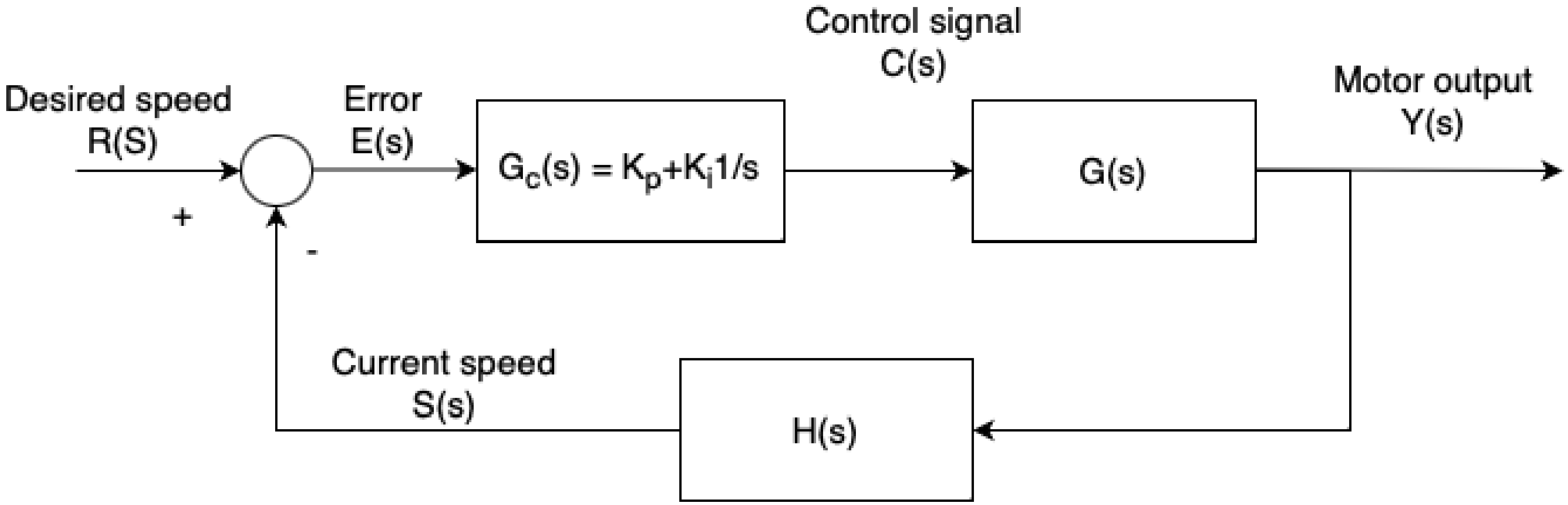
Integral term

- Integration computes area under the curve:



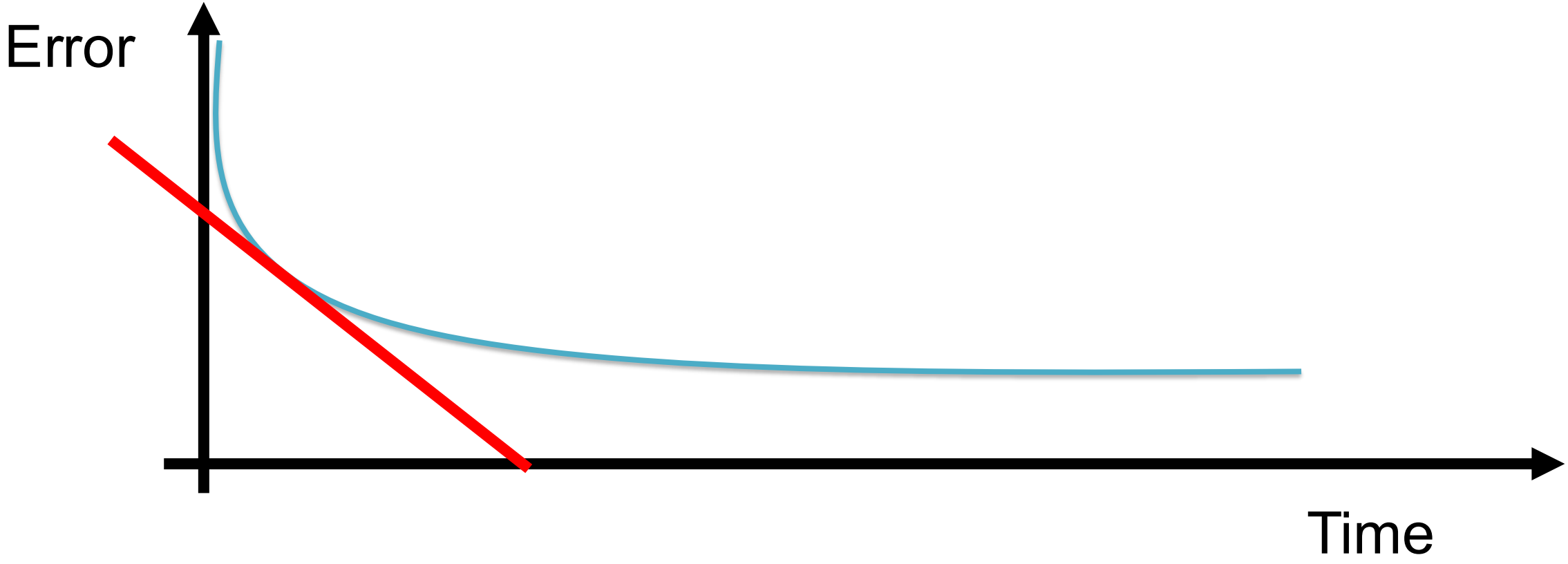
PI-Controller

- Integral allows us to eliminate steady state error
- Too much integral results in problem called ‘integral wind-up’
- The system becomes:



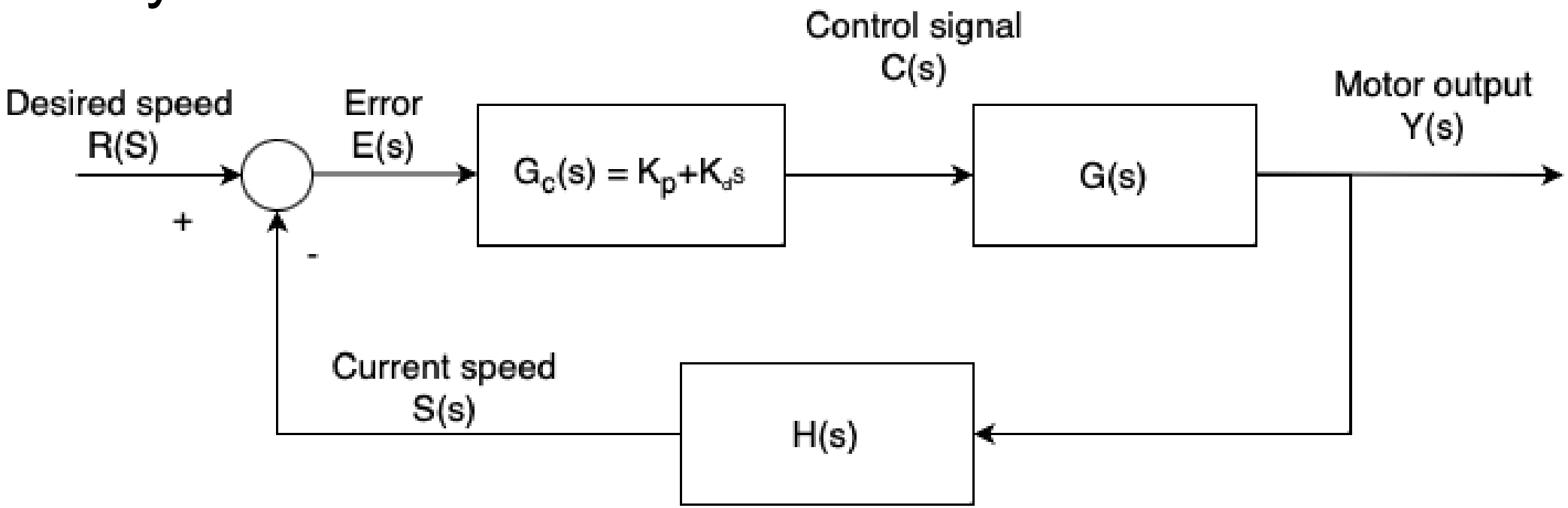
Derivative Term

- Derivation tells gradient (rate of change) of the error curve at any given point



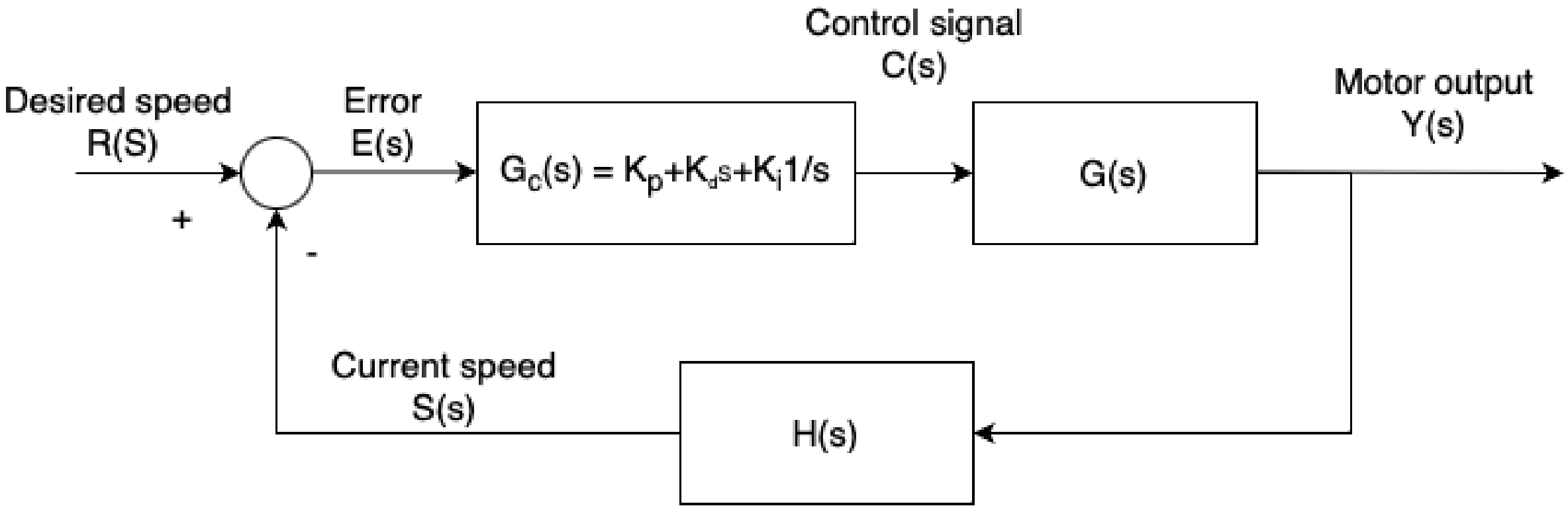
PD-controller

- Derivative term reduces overshoot – it checks for changes in gradient of the error
- The system becomes



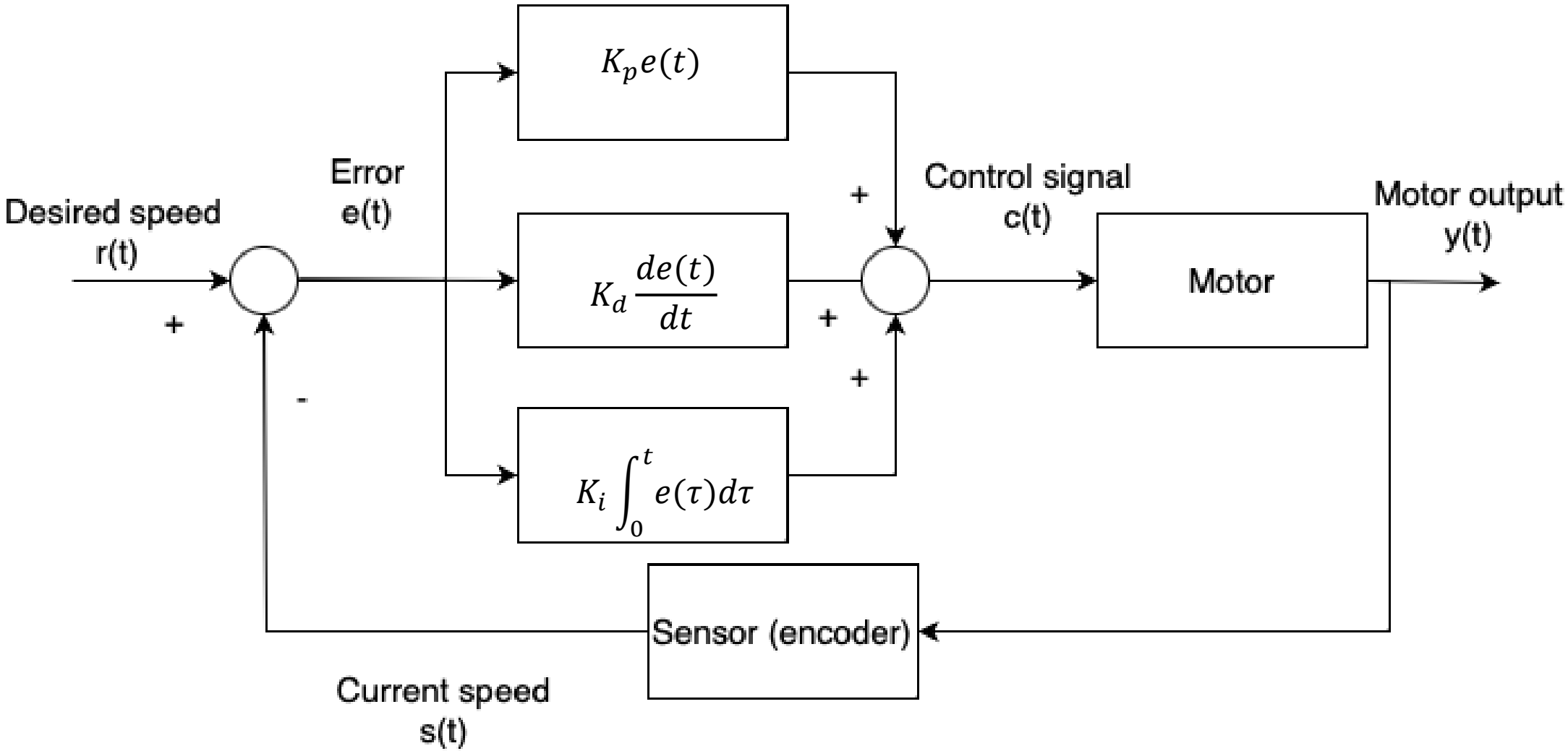
PID-controller

- Combine everything to get:



PID-controller in time domain

- In time domain we get:



MANCHESTER 1824
The University of Manchester

PID Tuning

Tuning guide

- The objective of the tuning is to find best K_p, K_I, K_D
- Most of the time we cannot model our plant exactly, thus algorithms for tuning are necessary
- Two main ones
  - Ziegler-Nichols
  - Manual
- None of the methods is bulletproof and is not guaranteed to work

Tuning guide

- (Rough guide to) effect of changing each parameter on performance

Parameter	Rise Time	Overshoot	Settling time	Steady-State error	Stability
K_p	Decrease	Increase	Minor effect	Decrease	Degrade
K_i	Decrease	Increase	Increase	Eliminate	Degrade
K_d	Minor decrease	Decrease	Decrease	No effect	Can slightly improve

- Note: those are only true when increased independently

Ziegler-Nichols

1. Set all gains to zero
2. Tune K_p until system starts to oscillate at a constant rate. That value is normally called K_u and oscillation period is known as T_u. Then set the values as follows:

Controller	K_p	K_i	K_D
PID	$0.60K_u$	$1.2K_u/T_u$	$3K_uT_u/40$

3. Tune slightly for better performance

Trial and error method

1. Set all gains to zero
2. Tune K_p until system starts to oscilate. Then half the K_p
3. Tune K_d until system starts to oscilate. Then half the K_d
4. Tune K_i until system starts to oscilate. Then half the K_i
5. Tune to desired performance

Other methods

- Cohen–Coon
- Relay method
- Auto-tuning
- … and many others

Derivative/Integrals in practice

- In practice computing derivative/Intgeral of error with time is complex/impossible
- Thus, approximations are necessary.
- Simplest approximation of derivative is:
- $\dot{e(t)} \approx \frac{e_n - e_{n-1}}{\Delta t}$
- Simplest approximation of integral is:
- $\int_{k=0}^{n} e(t) \approx \sum_{k=o}^{n} e_{0\ldots n}$

MANCHESTER 1824
The University of Manchester

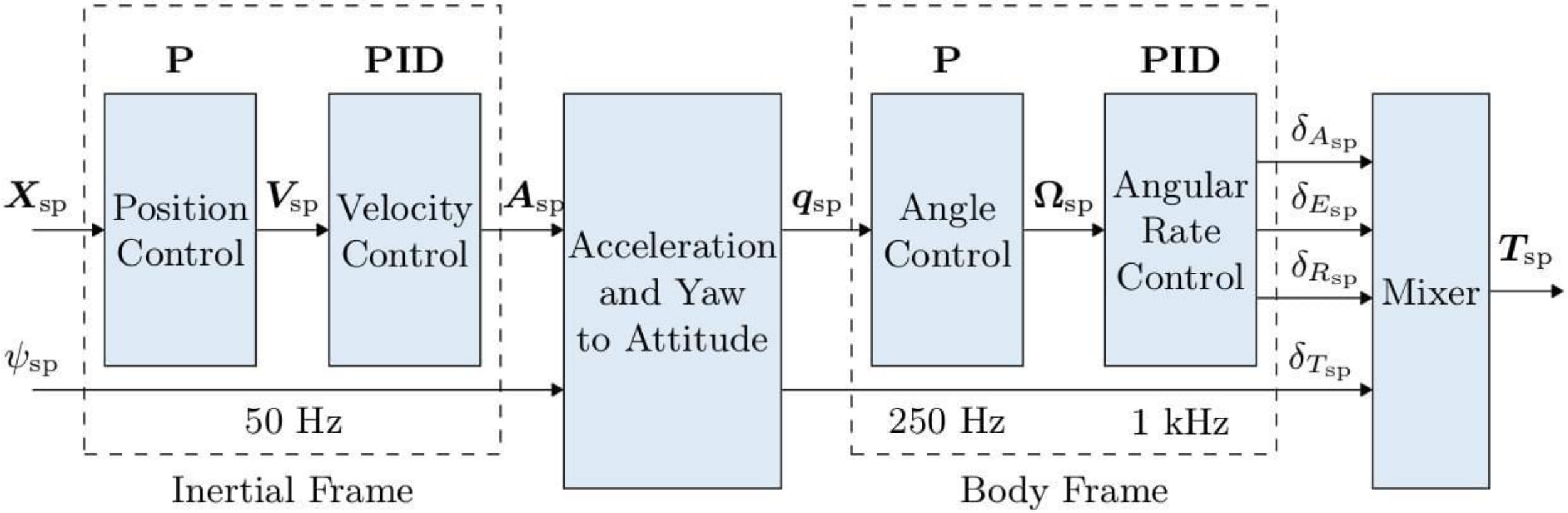
Advanced controllers (brief introduction)

Cascade Controllers

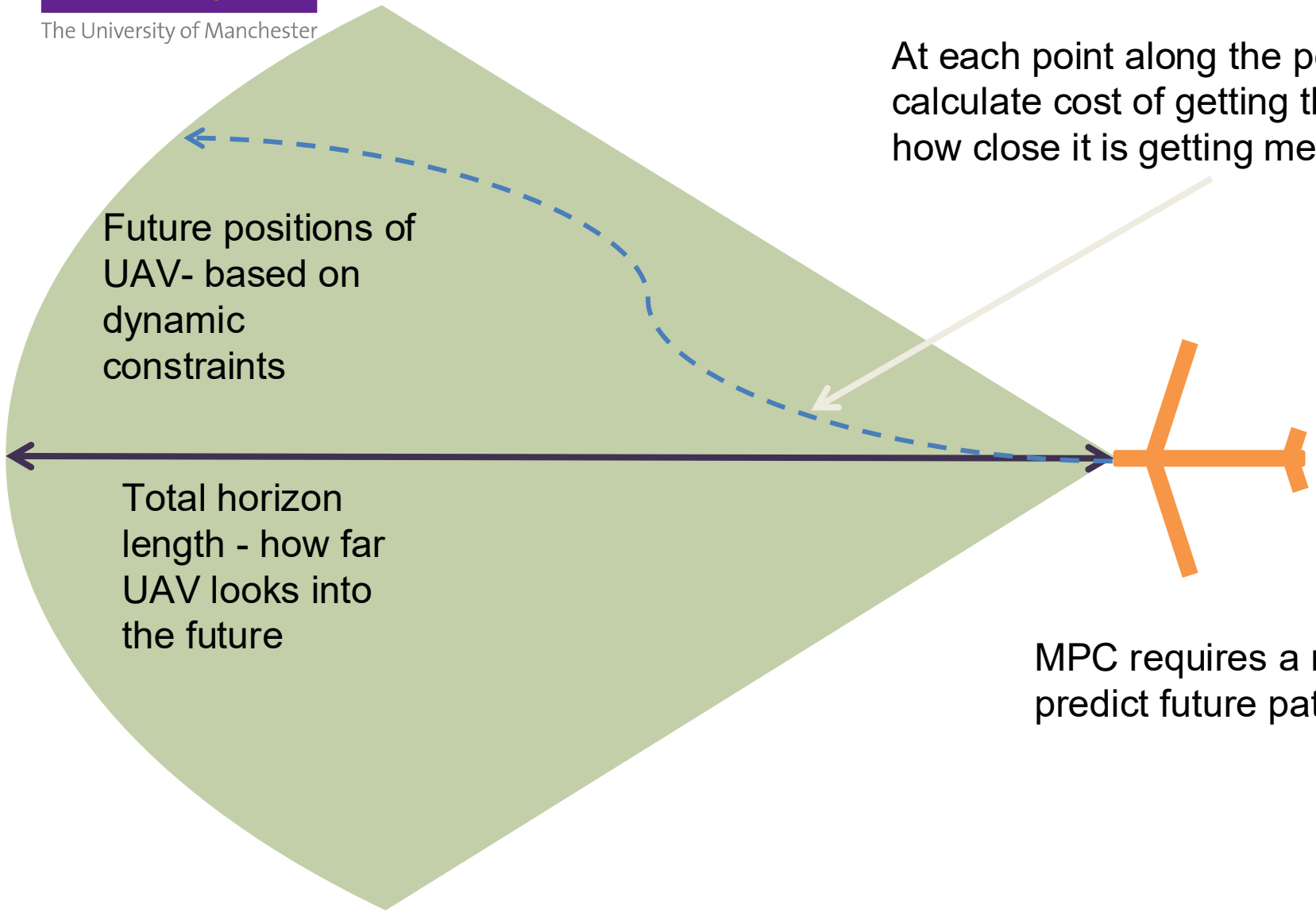
- Consider a UAV system where you have two sensors which are relevant to the process
  - For example, for UAV position control, we get positions update at 30 hz and attitude sensor updates at 250 hz
  - Note: to control the position you must control the attitude (and thrust) based on current position
  - But you can correct attitude a lot more often than position
  - Thus we can cascade controllers working one after another!

Cascade Controller example

– Example (from PX4 flight controller):



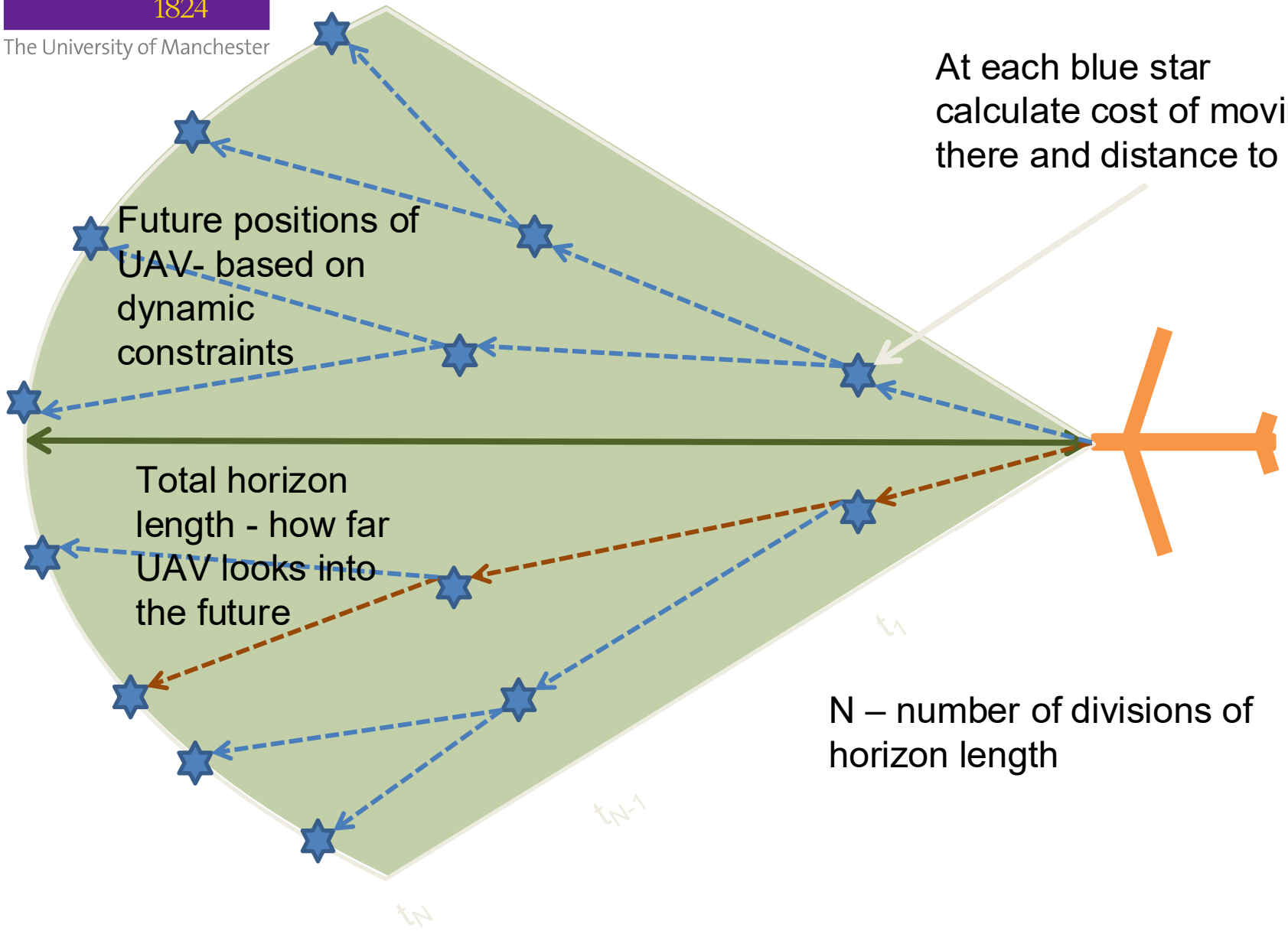
Model Predictive Control



At each point along the possible path calculate cost of getting there and how close it is getting me to the goal

MPC requires a model of our vehicle to predict future path

Model Predictive Control - Discrete



At each blue star calculate cost of moving there and distance to goal

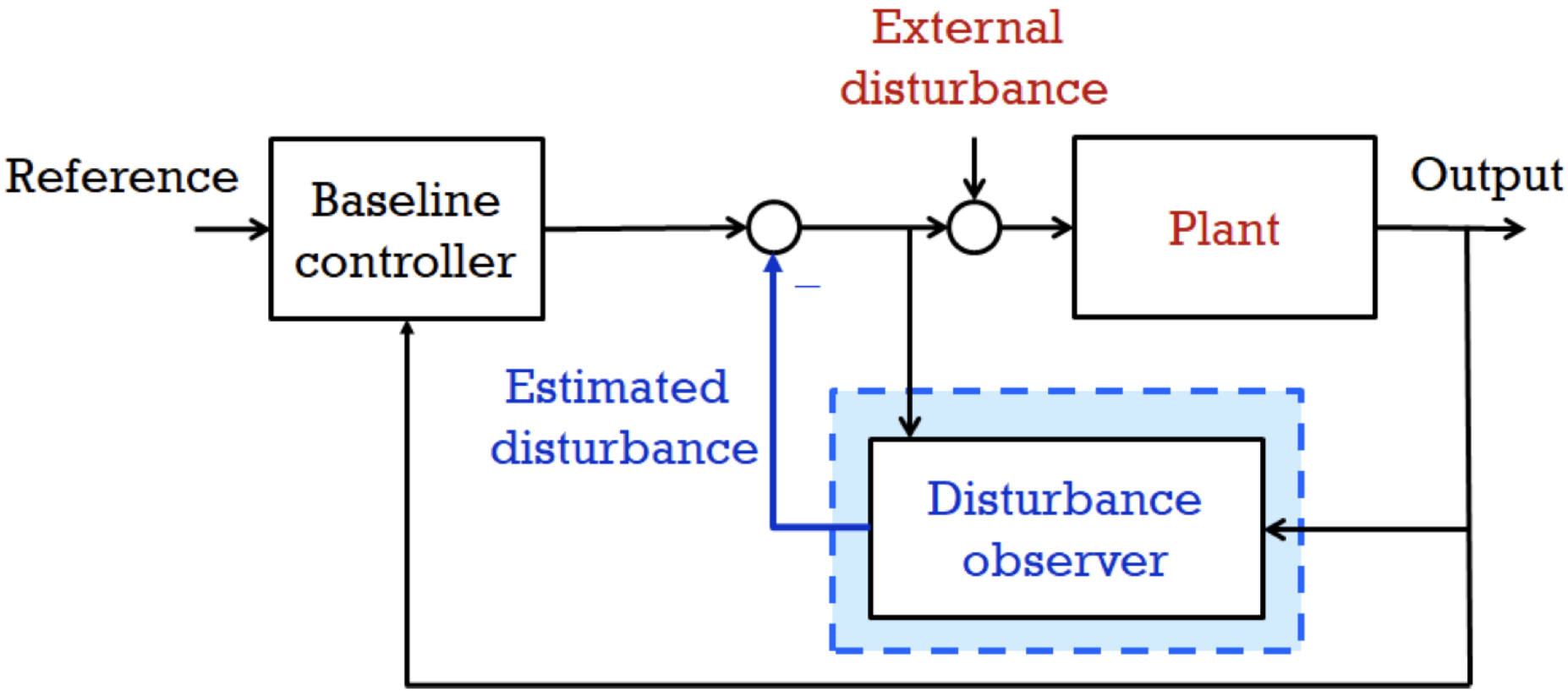
N – number of divisions of horizon length

Disturbance Observer Control

- Disturbance observer controller models the plant behavior
- Then it treats any deviation from the modelled behavior as disturbance and attempts to correct by strengthening control signal
- The plant model does not need to be extremely accurate

Disturbance Observer Control

- Using block diagram we get:



Reinforcement Learning

- Reinforcement learning can be used as a controller directly!
- Input for simple motor system would be current speed and desired speed
- Desired speed would be also reflected in reward function
- No complex modelling or tuning required, but training is needed

Summary

- The need for controller was discussed
- PID controllers were introduced and discussed
- Advanced methods were presented

MANCHESTER 1824
The University of Manchester

Thank you for listening and now onto activity

Activity

- Cascade PID tuning
- In groups of 2-3
- Please find classical inverted pendulum control problem here: https://vis-ro.web.app/robotics/pid (It also has small explanation of how the logic of tuning works)
- Use following parameters:
  - $m_c = 1.2$
  - $m_p = 2.3$
  - $l = 1.7$
- Try your best at tuning and post your parameters in the following menti: https://www.menti.com/ala1s6m949d2

Answers format

- Use comma separated values (see example on the left)

What are the k^θ_p, k^θ_d values?

1.231,1.21 130

What are the k^x_p, k^x_i, k^x_d values?

1.644,1.563,1.6332 122

Submit